



Universidad Politécnica del Centro

Carrera:

Ing. tecnologías de la Información e Innovación Digital

Docente:

Pablo Pérez López

Tema:

T1_U1 | Análisis técnico de stack móvil para proyecto real

Materia:

Desarrollo de Aplicaciones Móviles

Alumno | Matricula:

Ethan Ali Guzman Mendoza | 2505021

Cuatrimestre | grupo:

4ro | T3-1



1. COMPARATIVA DE FRAMEWORKS MULTIPLATAFORMA ACTUALES

Para la selección de la arquitectura cliente en la solución móvil de Gestión de Citas Médicas en iOS y Android, se evaluaron los tres entornos multiplataforma principales: Flutter, React Native (con Expo) y Kotlin Multiplatform (KMP).

Criterio	Flutter	React Native (Expo)	Kotlin Multiplatform (KMP)
Lenguaje	Dart	JavaScript / TypeScript	Kotlin
Rendimiento	Alto (60-120 FPS): Renderizado propio mediante el motor Impeller/Skia directo a canvas nativo.	Alto: Con la Nueva Arquitectura (Fabric + JSI), elimina el puente asíncrono JS.	Nativo Máximo: Compila a código binario nativo (JVM en Android, LLVM en iOS).
Curva de Aprendizaje	Media: Requiere aprender Dart y el paradigma reactivo basado en Widgets.	Baja-Media: Muy rápida para desarrolladores Web que dominan React, JSX y JS/TS.	Media-Alta: Exige dominar Kotlin y componentes UI nativos (Jetpack Compose / SwiftUI).
Comunidad y Ecosistema	Comunidad masiva. Gran cantidad de paquetes verificados en pub.dev. Respaldado por Google.	Ecosistema gigantesco (npm). Gran soporte mantenido por Meta y Expo.	En crecimiento acelerado. Respaldado por JetBrains y adoptado por Google y Netflix.
Ejemplos de Apps Reales	Google Pay, BMW App, Nubank, Alibaba, eBay Motors.	Meta (Instagram, Facebook), Shopify, Discord, Coinbase.	McDonald's, Netflix, Cash App, Forbes, Baidu.

Renderizado de UI	Propio (Widgets idénticos en todas las plataformas).	Componentes nativos del sistema operativo mapeados desde JS.	Nativo (UI separada en SwiftUI/Compose o Compose Multiplatform).
-------------------	--	--	--

Análisis Crítico para el Dominio Médico:

- **React Native (Expo):** Destaca por la rapidez de iteración mediante *Expo EAS*, despliegues OTA (*Over-The-Air*) para corrección rápida de errores en producción y facilidad para integrar paquetes de diseño accesibles.
- **Flutter:** Proporciona consistencia visual absoluta entre dispositivos de gama baja y alta, ideal para flujos interactivos de agenda y calendarios médicos.
- **KMP:** Excelente si se cuenta con lógica de negocio compartida compleja (cifrado local HIPAA/GDPR), pero incrementa los costos al requerir especialistas nativos para la capa de UI.

2. SELECCIÓN JUSTIFICADA DEL LENGUAJE DE PROGRAMACIÓN

Tras la evaluación comparativa, se selecciona TypeScript (sobre el ecosistema React Native / Expo) como el lenguaje principal del proyecto, superando a Dart, Kotlin y Swift por las siguientes razones:

1. **Tipado Estático Robusto y Prevención de Errores:** En aplicaciones médicas donde se gestionan expedientes, fechas de citas y recetas, los errores de tipo pueden comprometer la experiencia del usuario o la integridad de la información.

TypeScript permite definir esquemas estrictos de datos (interfaces para Doctor, Cita, HistorialClinico), detectando fallos en tiempo de compilación.

2. **Compatibilidad Full-Stack y Reutilización de Código:** La mayoría de las APIs de servicios de salud se desarrollan sobre Node.js, NestJS o servicios Serverless. Utilizar TypeScript en el cliente permite compartir tipos de datos, validaciones (ej. con bibliotecas como *Zod*) y lógica de negocio directamente entre el cliente móvil y el backend web.
3. **Disponibilidad de Talento y Mantenibilidad:** JavaScript/TypeScript es el ecosistema de programación con mayor disponibilidad de desarrolladores en el mercado local y global, lo que reduce costos de contratación y garantiza la continuidad del software a largo plazo.

3. ENTORNO DE DESARROLLO Y CONFIGURACIÓN

Herramientas a instalar:

1. **Visual Studio Code (VS Code):** El editor de código donde escribiremos la app.
2. **Node.js:** El entorno necesario para ejecutar las herramientas de JavaScript/TypeScript.
3. **Expo CLI:** La herramienta que nos ayuda a crear, probar y empaquetar la app de forma sencilla.
4. **Android Studio:** Necesario para instalar el SDK de Android y crear un celular virtual (emulador) para probar la app en la computadora.

Extensiones útiles para VS Code:

- **Expo Tools:** Ayuda a autocompletar comandos y archivos de configuración.
- **ESLint y Prettier:** Para que el código se ordene y corrija solo si nos equivocamos en alguna sintaxis.

Configuración básica del proyecto (package.json):

Este es el archivo donde se definen las librerías que usará la app (cámara, notificaciones, base de datos local):

```
C: > Users > ethan > Downloads > 4TO CUATRIMESTRE > Desarrollo de Aplicaciones Móviles > 1ra Unidad > [!] Gestion de Citas Medicas.json > ...  
1 {  
2   "name": "app-citas-medicas",  
3   "version": "1.0.0",  
4   "scripts": {  
5     "start": "expo start",  
6     "android": "expo start --android",  
7     "ios": "expo start --ios"  
8   },  
9   "dependencies": {  
10    "expo": "~51.0.0",  
11    "react": "18.2.0",  
12    "react-native": "0.74.1",  
13    "expo-camera": "~15.0.10",  
14    "expo-notifications": "~0.28.1",  
15    "expo-sqlite": "~14.0.3",  
16    "expo-local-authentication": "~14.0.1"  
17  },  
18  "devDependencies": {  
19    "@types/react": "~18.2.45",  
20    "typescript": "^5.1.3"  
21  }  
22 }
```

4. ANÁLISIS DE HARDWARE Y SENSORES DEL DISPOSITIVO

Una aplicación de citas médicas requiere interactuar estrechamente con el hardware del smartphone para brindar una experiencia ágil en clínica y garantizar la privacidad del paciente.

Componente / Sensor	Módulo / Librería Seleccionada	Caso de Uso en la App Médica
Cámara (Lector QR)	expo-camera / zxing	Check-in Express en Recepción: El paciente escanea el código QR en la sala de espera para confirmar su llegada y pasar a turno sin hacer filas.
Motor de Notificaciones Push	expo-notifications + FCM / APNs	Alertas y Recordatorios: Envíos programados 24h y 2h antes de la cita, alertas de cancelación de turno y confirmación de recetas médicas.
Almacenamiento Local (Offline)	expo-sqlite (SQLCipher)	Acceso a Citas e Historial sin Señal: Consulta de la agenda del día, datos del médico y mapa del consultorio incluso dentro de hospitales con mala cobertura.
Autenticación Biométrica	expo-local-authentication	Seguridad de Datos del Paciente: Acceso a la aplicación mediante Huella Dactilar (TouchID/Fingerprint) o Rostro (FaceID) para

		proteger el historial médico.
Calendario Nativo	expo-calendar	Sincronización de Agenda: Sincronizar automáticamente la cita confirmada con Google Calendar o Apple Calendar del usuario.

5. CONCLUSIÓN Y STACK RECOMENDADO

Después de analizar el rendimiento, la facilidad de desarrollo y las necesidades de una app médica, la combinación tecnológica recomendada es:

- **Framework de desarrollo:** React Native con Expo
- **Lenguaje de programación:** TypeScript
- **Base de datos en el celular:** SQLite (guardado local seguro)
- **Diseño e interfaz:** NativeWind (Tailwind CSS)

Justificación final:

Con esta combinación podemos programar la aplicación una sola vez y hacer que funcione tanto en celulares Android como en iPhone sin gastar el doble de tiempo ni dinero. Además, el uso de Expo nos resuelve fácilmente el acceso a la cámara, las notificaciones y la seguridad biométrica, mientras que TypeScript nos asegura que la información de los pacientes se maneje sin errores.



Universidad Politécnica del Centro



Universidad Politécnica del Centro